

Assignment No. 01

AI4003 - Applied Machine Learning

Fall 2026

Deadline: 15 September 2026

*Predicting Solar Power Plant Output from Weather
(least squares by normal equation and gradient descent, implemented by hand)*

Targeted Course Learning Outcome

CLO4: Design solutions using machine learning models with supervised or unsupervised techniques to address engineering challenges. (C6).

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.

Read the whole manual before you begin.

Contents

1	Overview	3
1.1	The problem	3
1.2	Dataset	3
1.3	Public weather data	3
1.4	Expected outcomes	4
2	Tasks and Instructions	4
2.1	Task 1 – Load and prepare the data	4
2.2	Task 2 – Exploratory plots	4
2.3	Task 3 – Download public weather data	5
2.4	Task 4 – Least-squares regression by hand	5
2.5	Task 5 – Analysis	6
2.6	Task 6 – Front end and communication	7
2.7	Required repository layout	7
3	Evaluation	7
3.1	Metrics	7
3.2	Correctness checks	8
4	Results	8
4.1	Table 1 – Data preparation	8
4.2	Table 2 – Test-set RMSE (kW)	8
4.3	Table 3 – Learned θ (Set A)	8
4.4	Figures	8
	Appendix: Data Loader	10

1. Overview

1.1 The problem

A solar plant's output depends on how much sunlight hits the panels and how hot they get. Plant operators measure this with on-site sensors, but a rooftop installer or a grid planner usually has no sensor, only a public weather service. In this assignment you will build a regression model that predicts a plant's hourly AC power output and answer one practical question:

The question

How much prediction accuracy is lost when the on-site irradiation and temperature sensors are replaced by free public weather data downloaded from the internet?

You will implement **linear regression** yourself with NumPy, exactly as derived in Lecture 1: the hypothesis $h_{\theta}(x) = \theta^{\top}x$, the least-squares cost $J(\theta)$, and three ways to minimise it – the **normal equation**, **batch gradient descent**, and **stochastic gradient descent**. No machine-learning libraries are allowed for modelling or evaluation.

1.2 Dataset

Solar Power Generation Data (Kaggle): two grid-connected photovoltaic plants in India, 34 days (15 May–17 June 2020), readings every 15 minutes.

Download <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>
Mirror <https://ieee-dataport.org/documents/solar-power-generation-data>

Four CSV files (about 2 MB total): one generation file and one sensor file per plant.

File	Column	Meaning
Plant_1_Generation_Data.csv	DATE_TIME	timestamp
Plant_2_Generation_Data.csv	SOURCE_KEY	inverter id (22 per plant)
	DC_POWER	DC power into the inverter (kW)
	AC_POWER	AC power out of the inverter (kW)
	DAILY_YIELD, TOTAL_YIELD	cumulative energy (kWh)
Plant_1_Weather_Sensor_Data.csv	DATE_TIME	timestamp
Plant_2_Weather_Sensor_Data.csv	AMBIENT_TEMPERATURE	air temperature (°C)
	MODULE_TEMPERATURE	panel temperature (°C)
	IRRADIATION	solar irradiation on the panel (kW/m ²)

Generation data is per inverter; sensor data is per plant. Every group works on **Plant 1**. A loader that reads the four files is given in the Appendix; everything after loading is your work.

1.3 Public weather data

Open-Meteo (<https://open-meteo.com>) provides free hourly historical weather for any coordinate from 1940 onward, with no API key. You will download it yourself with an HTTP request and parse the JSON response.

The exact coordinates of the plants are not published. Use the coordinates below as a starting point and *verify* them in Task 3.

	Latitude	Longitude	Time zone
Plant 1 (near Gandikota, Andhra Pradesh)	14.82	78.28	Asia/Kolkata
Plant 2 (near Nasik, Maharashtra)	19.99	73.79	Asia/Kolkata

1.4 Expected outcomes

1. Clean and merge real sensor data with inconsistent timestamps.
2. Collect data from a public web API and align it with a local dataset.
3. Implement least-squares linear regression via the normal equation.
4. Implement batch and stochastic gradient descent and choose a learning rate α .
5. Evaluate a regression model correctly on time-ordered data.
6. Deploy a model behind a simple interface and communicate the result.

2. Tasks and Instructions

Complete the tasks in order. Every task lists exactly what must appear in your repository.

Allowed libraries

`numpy`, `pandas`, `matplotlib`, `requests`, and a front-end framework of your choice for Task 6. **Not allowed anywhere:** `scikit-learn`, `statsmodels`, `scipy.stats`, or any other library that fits models or computes regression metrics. Matrix inversion with `numpy.linalg` is allowed.

2.1 Task 1 – Load and prepare the data

- 1.1 Download the dataset from Kaggle and place the four CSV files in `data/`. Load them with the loader in the Appendix.
- 1.2 Convert `DATE.TIME` in both Plant 1 files to a proper datetime column. Inspect the raw strings first; do not assume the two files use the same format. Print the first and last timestamp of each file after conversion.
- 1.3 Sum `AC_POWER` and `DC_POWER` over all inverters at each timestamp to get **plant-level** power.
- 1.4 Merge plant-level power with the sensor file on the timestamp. Report how many timestamps exist in only one of the two files.
- 1.5 Resample to **hourly means**. This gives about $34 \text{ days} \times 24 \text{ hours} = 816$ rows. Save as `data/plant1_hourly.csv` with columns `datetime`, `ac_power`, `dc_power`, `ambient_temp`, `module_temp`, `irradiation`.
- 1.6 Report: number of hourly rows, number of rows with missing values, and how you handled them.

2.2 Task 2 – Exploratory plots

Produce the following four plots from the hourly data. For each, write two or three sentences: what physics says the plot should look like, and where the data agrees or disagrees.

- 2.1 `ac_power` vs. `irradiation` (scatter).
- 2.2 `module_temp` vs. `ambient_temp` (scatter), coloured by irradiation.
- 2.3 `ac_power` vs. `dc_power` (scatter). Comment on the ratio.
- 2.4 Average `ac_power` for each hour of the day (line, 0–23).

2.3 Task 3 – Download public weather data

- 3.1 Using `requests`, call the Open-Meteo historical archive endpoint <https://archive-api.open-meteo.com> for Plant 1’s coordinates, start date 2020-05-15, end date 2020-06-17, time zone Asia/Kolkata, and hourly variables `shortwave_radiation`, `temperature_2m`, `cloud_cover`. Read the API documentation at <https://open-meteo.com/en/docs/historical-weather-api> to build the URL.
- 3.2 Parse the JSON response into a DataFrame with columns `datetime`, `sw_radiation`, `temp_2m`, `cloud_cover`. Save it as `data/plant1-openmeteo.csv`. Do this in a script, not by pasting the URL into a browser.
- 3.3 Merge it with `plant1_hourly.csv` on `datetime`. Note that Open-Meteo gives radiation in W/m^2 and the sensor gives kW/m^2 .
- 3.4 **Verify the location.** Plot sensor `irradiation` and downloaded `sw_radiation` on the same time axis for any three days. For each day, report the hour at which each curve peaks. If the peaks are shifted by an hour or more, the coordinates or time zone are wrong; investigate and fix before continuing.

2.4 Task 4 – Least-squares regression by hand

Notation (as in Lecture 1). For n training rows, $x^{(i)} \in \mathbb{R}^{d+1}$ is the feature vector of row i with $x_0 = 1$, $y^{(i)}$ is its `ac_power`, X is the $n \times (d + 1)$ design matrix with $x^{(i)\top}$ as rows, and $\vec{y}$ is the vector of targets. The hypothesis and cost are

$$h_{\theta}(x) = \theta^{\top} x = \sum_{j=0}^d \theta_j x_j, \quad J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

Data split (by date, no shuffling):

Train	15 May – 10 June	27 days
Test	11 June – 17 June	7 days

Two feature sets, each also including the time features $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$ where h is the hour of day, so $d = 5$:

Set A: on-site sensors	<code>irradiation</code> , <code>module_temp</code> , <code>ambient_temp</code>
Set B: public weather only	<code>sw_radiation</code> , <code>temp_2m</code> , <code>cloud_cover</code>

- 4.1 **Scale the features.** For each feature, subtract the training mean and divide by the training standard deviation. Apply the same numbers to the test rows. Gradient descent converges much faster when all features have similar scale.
- 4.2 Build X by adding the column $x_0 = 1$ (intercept) to the scaled features.
- 4.3 Write `hypothesis(X, theta)` returning $X\theta$, and `cost(X, y, theta)` returning $J(\theta)$.

4.4 Normal equation. Implement

$$\theta = (X^\top X)^{-1} X^\top \vec{y}$$

as `fit_normal(X, y)`.

4.5 Batch gradient descent. Implement `fit_batch_gd(X, y, alpha, n_iters)` starting from $\theta = \vec{0}$ and repeating, for every j ,

$$\theta_j := \theta_j + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}.$$

Record $J(\theta)$ after every iteration.

4.6 Stochastic gradient descent. Implement `fit_sgd(X, y, alpha, n_epochs)`: in each epoch loop over the training rows one at a time and update

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}.$$

Record $J(\theta)$ after every epoch.

4.7 Choose the learning rate. For batch gradient descent, run 500 iterations with $\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$ on Set A and plot $J(\theta)$ against iteration for the three runs on one figure. Identify which α is too small, too large, or about right, and use that one for the rest of the task. Do the same for stochastic gradient descent with $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and 50 epochs.

4.8 Fit all three methods on the training rows for Set A and for Set B (six fits in total). Confirm that batch gradient descent, given enough iterations, reaches the same θ as the normal equation (report the largest absolute difference between the two vectors).

4.9 Predict `ac_power` on the test rows with each fitted θ . Clip negative predictions to 0; a plant cannot produce negative power.

4.10 Report the test **RMSE** (formula in Section 3) for all six fits, on all test hours and on daytime test hours only (`irradiation` > 0).

2.5 Task 5 – Analysis

Answer in the blog and in `results/analysis.md`:

5.1 Print θ from the normal equation for Set A. Which feature has the largest weight, and does the sign of each weight agree with the physics from Task 2?

5.2 How much worse is Set B than Set A in daytime RMSE, in kW and as a percentage of the plant's peak hourly power? Is public weather data good enough for a rooftop installer?

5.3 Compare the three solvers: did they reach the same θ ? How many iterations or epochs did each need? Which would you choose for this dataset, and which for a dataset with 10 million rows? Refer to the “Optimization Decision Matrix” from Lecture 1.

5.4 Compare the batch and stochastic $J(\theta)$ curves. Which is smoother, and why?

5.5 Plot residuals $(y - h_{\theta}(x))$ against hour of day for the Set A normal-equation model. At which hours is the model worst, and what physical effect could cause it?

2.6 Task 6 – Front end and communication

- 6.1 A minimal web interface (Streamlit, Gradio, Flask, ...) where the user enters an hour of day and the three Set B weather values and receives predicted AC power from *your* saved weights. Screenshot in the README.
- 6.2 **Medium blog** (800–1500 words): the question, the data, the location check, the results table, and your answer to Task 5.2. At least two figures.
- 6.3 **LinkedIn post** linking the blog and repository.
- 6.4 **GitHub repository**: public; README with run instructions; meaningful commits from *both* members.

2.7 Required repository layout

data/	the four raw CSVs, <code>plant1_hourly.csv</code> , <code>plant1_openmeteo.csv</code>
src/load_data.py	the given loader
src/prepare.py	Task 1
src/eda.py	Task 2
src/fetch_weather.py	Task 3
src/regression.py	<code>hypothesis</code> , <code>cost</code> , <code>fit_normal</code> , <code>fit_batch_gd</code> , <code>fit_sgd</code> , <code>rmse</code>
src/train_eval.py	Task 4
results/	tables, figures, <code>analysis.md</code> , saved weights
app/	Task 6 front end
README.md	how to run everything

3. Evaluation

3.1 Metrics

Training: the cost $J(\theta)$ from Lecture 1, recorded during gradient descent.

Test: the root-mean-square error, which is the cost rescaled into kW so that it can be compared with the plant's actual power. With m test rows,

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)}))^2} = \sqrt{\frac{2 J_{\text{test}}(\theta)}{m}}.$$

Report RMSE twice: on all test hours and on daytime test hours only. Night hours are trivially predicted as zero and make every model look better than it is.

Expected range

Find the plant's peak hourly **ac_power** in the training data. With on-site sensors (Set A), daytime RMSE for a correct implementation is a small fraction of that peak, roughly 10% or less. With public weather only (Set B), expect it to be clearly larger. A daytime RMSE near half the peak for Set A means a bug; check timestamp alignment first. Normal equation and converged batch gradient descent must give almost identical RMSE.

3.2 Correctness checks

Before submitting, confirm:

- Scaling statistics come from training rows only.
- $J(\theta)$ decreases at every iteration of batch gradient descent with your chosen α .
- Batch gradient descent and the normal equation agree on θ to at least two decimal places.
- The test rows are never used for fitting or for choosing α .
- Predictions are clipped at zero.

4. Results

Present the following in **results/** and in the blog, using these layouts.

4.1 Table 1 – Data preparation

Raw generation rows (Plant 1)
Raw sensor rows (Plant 1)
Timestamps in only one file
Hourly rows after resampling
Hourly rows with missing values
Open-Meteo rows downloaded
Peak hour, sensor irradiation vs. Open-Meteo radiation (three days)

4.2 Table 2 – Test-set RMSE (kW)

Solver	Features	All hours	Daytime only
Normal equation	Set A		
Batch GD ($\alpha=$ __, __ iters)	Set A		
Stochastic GD ($\alpha=$ __, __ epochs)	Set A		
Normal equation	Set B		
Batch GD ($\alpha=$ __, __ iters)	Set B		
Stochastic GD ($\alpha=$ __, __ epochs)	Set B		

4.3 Table 3 – Learned θ (Set A)

	Normal eq.	Batch GD	SGD
θ_0 (intercept)			
θ_1 irradiation			
θ_2 module_temp			
θ_3 ambient_temp			
θ_4 sin hour			
θ_5 cos hour			
Max $ \theta_{\text{GD}} - \theta_{\text{normal}} $	–		

4.4 Figures

1. The four exploratory plots from Task 2.

2. Sensor irradiation vs. Open-Meteo radiation for three days (Task 3.4).
3. $J(\theta)$ vs. iteration for three learning rates, batch GD (Task 4.7).
4. $J(\theta)$ vs. epoch for three learning rates, stochastic GD (Task 4.7).
5. Actual vs. predicted AC power on the test week, normal equation, Set A and Set B.
6. Residuals vs. hour of day (Task 5.5).
7. Front-end screenshot.

Appendix: Data Loader

This is the only code provided. It reads the four CSV files as they are; it does **not** parse dates, merge, or clean anything.

A.1 Environment

```
1 pip install numpy pandas matplotlib requests
```

A.2 Getting the data

1. Create a free Kaggle account and open <https://www.kaggle.com/datasets/anikannal/solar-power-generation-data>.
2. Click **Download**, unzip, and place the four CSV files in `data/`.

A.3 `src/load_data.py`

```
1 from pathlib import Path
2 import pandas as pd
3
4 FILES = {
5     "gen1": "Plant_1_Generation_Data.csv",
6     "sensor1": "Plant_1_Weather_Sensor_Data.csv",
7     "gen2": "Plant_2_Generation_Data.csv",
8     "sensor2": "Plant_2_Weather_Sensor_Data.csv",
9 }
10
11 def load_raw(data_dir="data"):
12     """Return a dict of the four raw DataFrames.
13
14     Keys: 'gen1', 'sensor1', 'gen2', 'sensor2'.
15     DATE_TIME is left as a plain string on purpose.
16     """
17     data_dir = Path(data_dir)
18     frames = {}
19     for key, name in FILES.items():
20         path = data_dir / name
21         if not path.exists():
22             raise FileNotFoundError(f"Missing {path}. Download the dataset first.")
23         frames[key] = pd.read_csv(path)
24     return frames
25
26 if __name__ == "__main__":
27     raw = load_raw()
28     for key, df in raw.items():
29         print(f"{key}: {df.shape[0]} rows, {df.shape[1]} columns")
30         print(df.head(3).to_string(), end="\n\n")
```

Run `python src/load_data.py` once to confirm the files load. Look carefully at the printed `DATE_TIME` values before you begin Task 1.